

test → code → refactor

TDD en BarberSaaS

Desarrollo guiado por pruebas aplicado a nuestros microservicios

Sistemas Distribuidos · CORHUILA 2026-B · Carlos Leal

QUÉ ES TDD

Primero la prueba, después el código

Test-Driven Development: antes de escribir una función, escribimos la prueba que describe lo que debe hacer. El código existe solo para hacer pasar esa prueba.

EL CICLO

Rojo → Verde → Refactor

1 · ROJO

Escribo la prueba

Describe un comportamiento que aún no existe. Se ejecuta y falla.

2 · VERDE

Código mínimo

Lo justo para que la prueba pase. Nada de adornos.

3 · REFACTOR

Limpio el código

Mejoro nombres y estructura. Las pruebas confirman que nada se rompió.

POR QUÉ USARLO

Lo que ganamos

Diseño más simple

Si es fácil de probar, está bien desacoplado.

Refactor sin miedo

Las pruebas avisan al instante si algo se rompe.

Documentación viva

Cada prueba dice qué hace el sistema, y nunca se desactualiza.

Evidencia de calidad

Pruebas publicadas y corriendo en CI: lo que la norma puede verificar.

Todo empieza en la historia de usuario

Artefacto	Ejemplo
HU (04-requirements)	“Un cliente no puede reservar un horario ya ocupado”
Prueba	reservarHorarioOcupado_debeRechazar()
Código	Agenda.reservar(cita) valida el solapamiento
Pull Request	Referencia code-corhuila/barber-saas-docs#NN

Cada criterio de aceptación se vuelve una prueba. Así se cumple la trazabilidad ascendente (P.1).

TDD por capas en cada -api hexagonal

Capa	Qué probamos primero	Tipo de prueba
Dominio	Reglas: horarios, estados de cita, precios	Unitaria pura
Casos de uso	La orquestación, con puertos simulados	Unitaria con mocks
Adaptador REST	Que cumpla el OpenAPI de 07-api	Contrato
Persistencia	Repositorio contra el esquema del -db	Integración

De adentro hacia afuera: la mayoría de ciclos TDD ocurren en el dominio, sin base de datos ni framework.

Un ciclo sobre la agenda del barbero

1 · ROJO: la prueba (falla)

```
@Test
void reservarOcupado_debeRechazar() {
    var agenda = new Agenda(barbero);
    agenda.reservar(cita("10:00"));
    assertThrows(HorarioOcupado.class,
        () → agenda.reservar(cita("10:00")));
}
```

2 · VERDE: el código mínimo

```
public void reservar(Cita cita) {
    if (estaOcupado(cita.horario()))
        throw new HorarioOcupado();
    citas.add(cita);
}
```

3 · Refactor: por ejemplo, extraer estaOcupado() a un objeto Horario. La prueba sigue en verde.

Las pruebas también respetan la arquitectura

Regla 7

El esquema vive en el -db. Las pruebas de integración usan sus migraciones; el -api no crea tablas propias.

Regla 8

Si citas necesita datos de clientes, la prueba simula la API de clientes. Nunca toca su base.

Contratos

Las pruebas de contrato garantizan que esos mocks coinciden con el OpenAPI real de 07-api.

Más allá del -api

-workflow (sagas)

Primero las pruebas del flujo de 16-bpmn: camino feliz, fallo en cada paso y su compensación.

-worker

“Dado este evento, ocurre este efecto”. Y procesarlo dos veces no lo duplica (idempotencia).

-front y -app

TDD en la lógica y validaciones: “sin hora, muestra error”. En lo visual no compensa.

El ciclo queda visible en Git y en CI

```
test: reserva rechaza horario ocupado  
feat: validar solapamiento en Agenda.reservar  
refactor: extraer objeto Horario
```

El ci.yml de cada repo corre las pruebas en cada PR: si fallan, no se fusiona. El historial sirve como evidencia para el hu-status semanal.

CÓMO LO APLICAMOS

TDD en los 8 dominios, no solo en uno

- 01 Cada dominio convierte los criterios de sus HU en pruebas
- 02 TDD en el dominio y los casos de uso de cada -api
- 03 Pruebas de contrato entre dominios, según 07-api
- 04 Las pruebas corren en el ci.yml de cada repo: si fallan, no hay merge
- 05 Las mismas convenciones de pruebas en todos los dominios

**Si no hay una prueba que falle, no
hay código que escribir.**

¿Preguntas?